

Pramaan — Evidence Pipeline & Team Plan

What goes in, what happens to it, and who finishes what

What goes into the app, what happens to it, and who finishes what. One rule above everything else in this document: **every feature that ends up in the demo has to actually work, on real evidence, live.** No placeholder screens, no "this would work if," no pre-recorded fallback video. If something can't be made real in time, it comes out of the demo — it doesn't get faked.

Design ground rule still applies. Light theme only. DESIGN.md is dead.

1. What goes in, what happens to it, and why

1.1 What file do you actually give the app?

There are two kinds of input. They are not equivalent, and the app treats them differently.

A. A forensic disk image — the real input. This is a copy of the entire storage inside the DVR/NVR: .dd, .raw, .E01, or .ex01. Not a video file — a copy of the whole disk, byte for byte, the same way you'd image a laptop hard drive for a forensic case. This is what the problem statement actually asks for ("create forensic images"), and it's the only input type where the full pipeline — identify, parse, recover deleted footage — applies, because all of that only makes sense if you have the recorder's whole filesystem, not just a clip cut out of it.

What's actually inside a disk image like this:

- The recorder's own filesystem/index — a table the recorder itself maintains, listing every recording it knows about: which channel (camera), start time, end time, and whether that slot is still valid or has been marked for reuse ("deleted"). Different vendors store this differently (Hikvision's HIKBTREE, Dahua's DHFS), but every vendor has something like it.
- The actual video data for every recording still physically present on the disk — including recordings the index says are "deleted." DVRs work like a circular buffer: when disk space runs out, the oldest recording's index entry gets marked as reusable and new footage gets written over it eventually. Until that overwrite actually happens, the old video bytes are still sitting on the disk, just no longer listed as valid. That's what "recovering deleted footage" means here — reading data that's still physically there but that the recorder itself no longer shows you.
- Sometimes: system event logs (e.g. Hikvision keeps a separate log region with motion/alarm events), which is extra metadata beyond just the recording list.

B. An already-exported video clip — the shortcut path. .mp4, .avi, .dav, .264, etc. This is what you get if someone already pressed "export" on the DVR itself, or handed you a clip pulled off the device some other way. It contains only the video frames for that one clip — no index, no deleted recordings, because those only exist on the full disk. The app still accepts this (hash it, analyze it, run it through cross-camera trace), but it skips identification and recovery entirely, because there's no filesystem here to identify or recover from. This path is a convenience, not the graded forensic workflow — don't present it as the same thing in a demo.

1.2 Metadata comes out first, before anything is touched

This is the part that was confusing, so here it is plainly: **the app reads metadata straight from the original disk image, before it creates any new file.** It does not extract metadata, then convert the video, then somehow lose or reconstruct that metadata. The order is:

1. Read the recorder's index directly off the disk image (still just reading, nothing written or converted yet). This gives channel number, start/end time, and deleted-or-not — for every recording, without decoding a single frame of video.
2. For one specific recording, go to its byte range on the disk image and pull out the video data sitting there.
3. Only now, for that one recording, strip the vendor's wrapper and write a new, ordinary .mp4 file so it can actually

be played and analyzed.

Steps 1 and 2 never modify or discard anything — they're reading the same untouched disk image the whole time. Step 3 is the only place a new file gets created, and by then the metadata that mattered (channel, timestamps, deleted status, and the hash of the original image) has already been pulled out and saved in the case database, tied permanently to that recording. The new .mp4 file doesn't need to carry that metadata in its own header — an ordinary video file format has no field for "recorder allocation state" anyway. The database is where that metadata lives; the .mp4 is just something a person or an AI model can actually open.

So why make the new file at all, if the metadata's already safe? Because nothing else can use the recorder's raw video data directly. A vendor's on-disk video block is wrapped in that vendor's own private format — no ordinary video player can open it, and the app's own AI analysis (which decodes frames with OpenCV) can't read it either. Producing one ordinary .mp4 is what lets a human watch the footage and lets the AI pipeline analyze it. It's a convenience copy for consumption, not a replacement for the original evidence — the original disk image stays untouched and is still the thing that's hashed, cited, and treated as evidence in the report.

The only twist: turning the vendor's wrapped video into a normal .mp4 should never re-encode the actual video (decode it and compress it again) if it can be avoided, because re-encoding is lossy and slow. It should copy the underlying video bytes as-is into a new container — same picture, same quality, just repackaged. Re-encoding is a last resort, only when the data is too damaged to copy cleanly, and it must be logged so nobody mistakes it for a clean copy.

1.3 Walkthrough — a seized NVR, start to finish

A shop is robbed. Police seize the shop's NVR (the recorder box) and hand the examiner its hard drive.

1. **Acquire.** The examiner connects the drive through a write-blocker and images it: every byte, copied into one file, `shopcase_nvr.E01`. Pramaan hashes this file (MD5 + SHA-256) the moment it's registered. This file is never modified again for the rest of the case — every later step reads from it, nothing writes back to it.
2. **Identify.** Pramaan looks at the first chunk of that image, finds the signature that says "this is a Hikvision recorder, HIKBTREE filesystem," and reports that with a confidence score, not a guess dressed up as certainty.
3. **Parse.** Pramaan walks the HIKBTREE index inside that same image (still hasn't created a single new file) and produces a list: Channel 1, 09:00–09:45, valid. Channel 1, 09:45–10:15, marked deleted. Channel 2, 09:00–10:15, valid. This list is most of what an investigator actually wants to know, and it came from reading index bytes only — no video has been touched yet.
4. **Recover.** For the deleted 09:45–10:15 entry, Pramaan goes to that byte range in the image and checks whether the video data is still there. It is (nobody's recorded over it yet), so it gets carved out. If it had been partially overwritten, Pramaan would recover only the part that's genuinely intact and mark the rest as unrecoverable — never guess at the missing part.
5. **Unwrap and repackage.** That carved-out data is still wrapped in Hikvision's own format. Pramaan strips that wrapper, finds the H.264 video underneath, and copies it — not re-encodes it — into an ordinary file: `channel1_0945-1015_deleted.mp4`. This is a new file. The original `shopcase_nvr.E01` is untouched.
6. **Hash the new file too.** This .mp4 gets its own hash, separate from the image's hash. The case record now says, in effect: this mp4 (hash X) came from `shopcase_nvr.E01` (hash Y), channel 1, byte offset Z, recorder said 09:45–10:15, the index had marked it deleted. Every fact traces back to where it actually came from.
7. **Normalize the timestamp.** The recorder's clock might be off — DVRs left un-synced drift over time. If the examiner has calibrated that device's drift, the corrected time gets recorded alongside the recorder's original time (both are kept, not one overwriting the other).
8. **Now the file is usable.** The examiner can play `channel1_0945-1015_deleted.mp4` in any ordinary video player. Pramaan's own analysis can decode it frame by frame looking for people, objects, or motion. If footage from Channel 2 covers an overlapping time window, cross-camera trace can compare the two for the same person.
9. **Report.** The final PDF cites the image's hash, the recording's hash, the byte offset it came from, the recorder's original timestamp and the corrected one, and the fact that this specific recording was recovered from a deleted index entry. It doesn't just show a video — it shows exactly where every fact in that entry came from.

That's the whole pipeline. Everything else in the app — cross-camera trace, findings, the report — operates on the outputs of steps 6–9, never on the original image directly.

2. Three scope decisions

Corrupted footage. Recover what's genuinely there. Bytes that are missing or overwritten get marked PARTIAL — never filled in with a guess. This is already how the code behaves (`recovery_status`); it has to stay that way everywhere, including anything new that gets built.

Upscaling or "enhancing" video — deliberately out of scope. It's not in the problem statement, and it shouldn't be added. AI super-resolution invents pixel detail that was never actually captured — a real risk in a forensic tool, not a nice-to-have feature, because it's exactly the kind of thing that gets video evidence challenged in court. If a judge or teammate raises it: Pramaan is one tool in a forensic workstation, the same way commercial tools like DVR Examiner or UFS Explorer are — recovery, parsing, and hashing here; a dedicated video-enhancement tool is a separate job for a separate tool. That's a stronger answer than trying to build a "safe" upscaler.

Cross-camera trace runs on the derived clips, not the original image. Person matching and face matching need to decode video frames, and that only works on the ordinary .mp4 produced in step 6 above, not on the raw disk image. That's expected and fine — it's exactly the "one place lossy analysis legitimately happens" from section 1.2, and every match still cites back to which recording, and therefore which hash, it came from.

3. Where things stand — spot-checked directly against the code, not a full re-audit

Area	Status	How I know
Cross-camera trace, Findings proximity flags	Shipped	Built and merged this session (Navneet).
Live devices	Removed entirely	Confirmed out of PS scope; deleted frontend/backend/tests/CI job.
Hikvision parser (<code>hikvision_fs.py</code>)	Solid	Cites 2 peer-reviewed papers + 1 corroborating open-source project; tracks deleted as a distinct state, not a guess.
Hikvision recovery on the current lab image	Fixed today	Root cause was <code>build_oem_disk_fixtures.py</code> truncating the specimen to 4 MiB, cutting off its HIKBTREE entries — not a parser bug. Fixed, fixture regenerated, verified through the real acquire -> identify -> recover path: 5 recordings, one correctly marked deleted. See Aravind's section.
Dahua recovery on the real .dav	Works	Ran it today end to end: acquire -> identify (0.98 confidence) -> recover -> 1 segment recovered -> playable.
Dahua DHFS parsing	Still carve-only	<code>dahua_dhfs.py</code> scans for the DHAV frame signature and treats a 4KB gap as "deleted." It never reads the DHFS index the way Hikvision's parser reads HIKBTREE.
Recovery page deleted column/filter	Present	<code>deletedOnly</code> toggle + allocation glyphs exist in <code>CaseRecoverPage.tsx</code> .
Ranged export (<code>-ss/-c</code> copy)	Verifies before trusting	Checks the result before accepting it, falls back to re-encode on failure — the old silent-empty-file bug looks fixed.
Report HTML preview	Still dark-themed	<code>reporting.py</code> still sets <code>background:#0a0e1a</code> — the one screen where the rejected dark UI is still live.
Job log	Still recovery-only	Only filters <code>job.kind === "recovery"</code> — analytics, export, and cross-camera jobs are invisible in it.
Settings	"Parser sanity check" panel still present	Still shown as a product feature; it's a test-suite runner and belongs in CI, not the app.
Evidence catalog	"Lab specimen" category still present	Still a real, selectable filter in the UI.

Area	Status	How I know
Blockchain & Cybersecurity theme fit	Already true, not said out loud	The custody log is an append-only, hash-chained record — that already is the "tamper-evident ledger" the theme is asking for. Nothing in the report or pitch currently says this.
BSA Section 63 certificate format	Missing	reporting.py has no wording tied to Section 63 or a named responsible official — the report isn't shaped like the specific legal certificate format Indian courts look for.
Clock-drift warning	Missing	drift_offset_seconds silently defaults to 0 when nobody's calibrated it. No warning is shown when a device's clock was never checked.

4. Who does what

Split by real effort, not how many bullet points each person has — one hard reverse-engineering bug can take longer than five small fixes put together. If anyone's list turns out heavier than it looks once they're in it, say so — the two smallest, most self-contained items on Jai Pranav's list (removing the Settings panel, removing the Evidence catalog category) can go to whoever finishes their own list first. That's the release valve; use it instead of quietly falling behind.

Every item below ends the same way: working on real evidence, demonstrated live, not shown as a recording or described as "should work."

Navneet — Cross-camera trace (own scope)

1. Get real multi-angle footage, not just multi-time-window footage. The current demo proves the matching pipeline (build short tracks per source -> group tracks across sources by appearance similarity -> let someone search by photo) genuinely works across two separate video files, but the two "cameras" used were two overlapping time windows cut from one real camera feed — same angle, same lens, same lighting, the whole time. That's real machinery tested on an easy case. A different physical camera looking at the same person from a different angle, distance, and lighting is the actual hard case this feature exists for, and it hasn't been tried yet.

- How to get it without needing real CCTV hardware: film the same short walk (10–20 seconds is enough) twice, from two clearly different positions — two phones, or a phone and a laptop webcam, works fine. Match real CCTV conditions as much as possible: shoot from a few meters away, not a close-up portrait, since the appearance model was tuned on 1600x1200 NVR footage, not phone selfie framing.
- Register both clips into a case as evidence, run a correlation, and open the identity that should represent that person. If the model links the two sightings correctly, note the actual similarity numbers achieved (for the report/demo talking point — replace the current numbers, which came from the same-camera test, with real cross-angle ones). If it doesn't link them, that's a real result too — document it plainly (the appearance model may simply not be robust to a real viewpoint change) rather than quietly avoiding that test before the demo.
- Try it at more than one sensitivity setting if the first attempt doesn't link correctly — the match-strictness slider on the run-setup panel controls exactly this.

2. Add a plain verdict next to each match, not just a similarity number. Right now search_person in engine/app/services/cross_camera.py returns a bare cosine-similarity float per match, and the frontend (FindPanel in CaseCrossCameraPage.tsx) just prints it as a percentage bar. Two pieces of data needed for a verdict are already computed and sitting in the database for every appearance — the similarity score, and each appearance's timestamp — so this is a rule layered on existing data, not new data collection:

- In search_person, alongside similarity, compute the time gap between the query context and the matched appearance (or, for the identity-level "movement across cameras" view, the gap between last-seen-on-camera-A and first-seen-on-camera-B for the same identity).
- Apply a simple, stated rule: similarity above some floor (start from the thresholds already used for clustering, 0.40–0.75 depending on strictness) and a plausible time gap -> label it a **strong match**; similarity present but the gap is large or uncertain -> **review manually**, with the reason shown, not just a color.
- This doesn't require knowing the physical distance between cameras (which the app has no way to know) — it only needs to stop presenting a bare number and start presenting a stated judgment call, which is what makes a match

defensible under a follow-up question instead of just asserted.

- **Done when:** every cross-camera match shown in the UI (both the Find-a-person results and the identity movement view) carries a verdict and a one-line reason, not just a number.

3. Rehearse the evidence-trail moment as part of the actual demo script, not something left for someone to stumble on: click a detection -> open the frame -> Save as evidence -> watch the sha256 appear. This already works end to end; it just needs a place in whatever gets said out loud during the demo, since it's the direct answer to "how do you know this AI match traces back to real evidence."

4. Optional, and only after checking with Roshan first: the problem statement's own phrase is "correlate events across cameras," which is broader than matching one person's appearance. Lining up Findings' motion/object detections by timestamp across channels on Timeline's shared axis (once that axis is actually shared — see Roshan's list) would answer that more completely. Don't start this if Roshan's work already covers it — check first, since building the same alignment logic twice wastes both of your time.

Aravind — Hikvision recovery

The 0-recordings bug is already root-caused and fixed — verify it, then move to the harder remaining problem.

What was actually wrong: HikvisionAdapter.list_recordings() in engine/app/parsers/hikvision.py and the HIKBTREE-walking code it calls in engine/app/parsers/schemas/hikvision_fs.py were never broken — a unit test that builds a specimen fresh in memory and parses it (engine/tests/test_m2_parsers.py) was passing the whole time. The actual bug was in scripts/validation/build_oem_disk_fixtures.py: its build_hikvision_disk() function wrapped the specimen in a fixed 4 MiB "disk" and **truncated anything longer to exactly 4 MiB**. The specimen builder (engine/app/verification/hikvision_specimen.py) has grown past 4 MiB since that truncation was written, so every recent regeneration of lab_hikvision_fs.img silently cut off the HIKBTREE entries near the end of the file — confirmed directly: rebuilding the specimen and feeding it to the parser without truncation returns 5 recordings (one correctly marked deleted); feeding it through the old truncate-to-4-MiB logic reproduces exactly 0. Fixed by changing that function to never truncate, and regenerated all three OEM lab fixtures through the real script. Verified through the actual HTTP path, not just the parser directly: acquire -> identify (Hikvision, HIKBTREE match) -> recover now returns 5 segments across 2 channels on a clean engine instance.

With that resolved, what's actually left is the harder, more important gap:

- **Independently re-run the fix.** Pull latest, run python scripts/validation/build_oem_disk_fixtures.py, then acquire the regenerated lab_hikvision_fs.img through the app (not just the API directly) and confirm recovery still shows 5 recordings with one deleted. Don't take this write-up's word for it — it's your area, verify it yourself.
- **The real gap: nothing in this parser has ever touched a real Hikvision drive.** Every offset in docs/reference/hikvision_fs.md rests on two published papers plus one open-source project's corroboration — solid sourcing, but still second-hand. If a real (even old, even cheap secondhand) Hikvision DVR/NVR can be sourced, record a short test clip on it, delete it through the recorder itself (not by hand), then image the drive and run it through identify -> recover. If every offset in the reference doc lines up and the deleted clip comes back, that's the first real-hardware validation this parser has ever had — genuinely worth stating plainly in the demo ("validated against a real device," not just "validated against research papers").
- **If a real device can't be sourced in time, harden the synthetic path instead of leaving it as one flat case.** hikvision_specimen.py currently builds one clean, small HIKBTREE with a handful of entries. Extend it (or add a second specimen variant) to cover the structural edge cases a real drive is more likely to hit: entries spread across more than one HIKBTREE index page (the page-walk loop in schemas/hikvision_fs.py has a hard safety cap at 100 pages — worth actually exercising that path, not just trusting it), and a recording whose data block has been partially overwritten (confirm it comes back PARTIAL, not silently dropped or silently treated as complete).
- **Done when:** the regenerated fixture is confirmed working by you independently, and either a real device has validated the parser once, or the synthetic fixture has been extended to cover multi-page and partial-overwrite cases with a test proving each one behaves correctly.

Aslam — Dahua filesystem and the video-repackaging step

Building real DHFS index parsing, not the current magic-byte scan. Where to start: docs/reference/dhav.c is FFmpeg's actual DHAV demuxer source, already sitting in this repo as a reference — it's the most authoritative public source for the container's structure. docs/reference/dhav_fixture_spec.md documents the synthetic fixture layout already

used for testing. The synthetic test target is `validation_data/oem/lab_dahua_dhfs.img` — built by `scripts/validation/build_oem_disk_fixtures.py's build_dahua_disk()`, which places a DHFS4.1 signature in the first 1024 bytes and embeds the specimen at byte offset 512×1024 . That gives you a known, controllable starting point to develop against before ever touching the one real Dahua file in the corpus.

- Current state: `engine/app/parsers/dahua_dhfs.py` streams through the whole image looking for the DHAV frame signature and stitches contiguous frames together — it never reads an index at all, so it has no real concept of "this byte range is an allocated recording" versus "this byte range is empty." Its current stand-in for "deleted" is just any 4 KB gap between two carved frames, which isn't reading anything the recorder actually wrote — it's a guess.
- Build a real index reader the same shape as Hikvision's: locate the DHFS4.1-marked index region, walk its entries (channel, start/end timestamp, allocation status), and use that to enumerate recordings instead of scanning for frame headers. Keep the current magic-byte scan as an explicit, clearly-labeled fallback for when the index itself is damaged or absent — that's a legitimate secondary path, just not the primary one.
- Once the index reader exists, "deleted" becomes a real check: a recording is deleted when its byte range falls outside anything the index currently claims as allocated, the same logic Hikvision's parser already uses for its deleted (index entry cleared) state.
- Validate the same way Aravind's fix was validated: build the fixture fresh with `build_oem_disk_fixtures.py`, run it through the app's real acquire -> identify -> recover path (not just a unit test calling the parser directly), and confirm the reported deleted count matches what was actually built into the fixture.
- The ranged-export ffmpeg fix from the last audit round was verified against Hikvision-style carved data; re-verify it specifically against a Dahua recording once the index reader is producing real byte ranges to export, since it's a different code path through `api/v1/devices.py's export endpoint`.
- **Done when:** Dahua recovery reports a deleted count sourced from reading the DHFS index, not from a gap heuristic, verified through the real app pipeline on the synthetic fixture, and ideally cross-checked against the one real Dahua .dav file already in the corpus (`dahua_19.25.00-19.25.50-R.dav`) to confirm the frame-level parsing still agrees with real data.

Roshan — Timeline and playback across channels

The shared-axis gap is confirmed, not just suspected — here's exactly where. In `src/components/TimelineView.tsx`, the code that computes each channel's time range (min, max, span, and the tick marks built from them) sits inside the `channels.map(...)` loop that renders each channel's lane — meaning every channel currently computes and draws its own independent time axis, scaled to its own earliest and latest recording. Two channels covering different but overlapping real time ranges will each stretch to fill their own lane width, so their tick marks and bar positions don't actually line up against each other even though they're stacked visually. The fix is straightforward once you know where to look: compute one shared min/max/span/tick-list across every channel's segments before the per-channel loop, then have each channel's lane use that same shared scale instead of computing its own.

- Build (or reuse) a 2-channel test case — the two demo clips (`demo_camA.mp4` / `demo_camB.mp4`) or the real Dahua .dav alongside a second registered clip both work — and confirm visually, after the fix, that the tick marks match between lanes and that dragging the playback position on one channel's lane moves the position indicator on the other channel's lane too (`PlaybackDeck.tsx` is where the shared playhead logic lives).
- **Deleted recordings need a visibly different color, a legend, and a "deleted only" toggle** in `TimelineView.tsx`, driven by the real `allocation_state/deleted` field Aravind's and Aslam's parsers now report through `list_recordings` — not a placeholder or a guessed color.
- **Small addition: a visible warning when a device's clock was never calibrated.** `drift_offset_seconds` (used in `timeline.py` and passed into `PlaybackDeck` as `driftOffsetSeconds`) silently defaults to 0 when nobody's run drift calibration for that device — which looks identical, on screen, to "we checked and it's perfectly synced." Those are two very different forensic claims and they need to look different. A device whose drift was never calibrated should say so plainly wherever its timestamps are shown.
- **Done when:** a 2-channel case shows both lanes sharing one time axis with matching tick marks, moving one channel's playback position visibly moves the other, deleted recordings are visually distinct with a working filter, and an uncalibrated device's timestamps carry a visible warning instead of silently reading as verified.

Jai Pranav — Report, Job log, Settings, Evidence catalog, and pulling it all together

- **Report — restyle it.** The generated report (engine/app/services/reporting.py) still sets background:#0a0e1a in its HTML output — a straight find of that hex value and the surrounding style block is where to start. Replace it with the same palette every other screen already uses: white surface, near-black text, one blue accent. This is the last screen in the whole app where the rejected dark theme survives.
- **Report — shape the signing section like a real BSA Section 63 certificate.** Nearly all the data this needs already exists elsewhere in the app — device make/model/serial from Identification, the image and recording hashes from Acquisition and Recovery, the examiner/actor name from the custody log — so this is mostly a template and layout task, not new data collection. Lay out one clearly-labeled section with: device details, both hashes (image and derived clip), how the evidence was extracted (which adapter, which job), who performed the extraction, and a blank, clearly-marked line for a named responsible official's signature. The tool fills in every field it can attest to; it does not sign on a human's behalf — that distinction is what makes this legally honest rather than overreaching.
- **Job log — show every kind of job.** CaseJobsPage.tsx currently only surfaces rows where job.kind === "recovery"; analytics, export, and cross-camera jobs exist in the same jobs table but never appear here. Add a visible job-kind column and stop filtering them out.
- **Settings — remove the "Parser sanity check" panel.** It's a pytest-style regression suite wearing a "Run" button, shipped as if it were a product feature (SettingsPage.tsx plus its backend in tool_verification.py / tool_verification_report.py). Real test coverage belongs in the automated suite that already runs in CI, not in a button end users would see.
- **Evidence catalog — remove the "Lab specimen" category.** CaseEvidenceCatalogPage.tsx still has a real, selectable filter and icon for it — a leftover from earlier synthetic-data testing that has no place as a real option in a finished tool.
- **Pull it all together.** scripts/demo/mvp.sh still has a step that assumes Live devices exists — update or remove it now that the menu is gone. Then actually run the full script end to end, on the real Dahua .dav, and confirm every step from Section 1 (acquire -> identify -> parse -> recover -> unwrap -> repackage -> normalize -> hash -> analyze -> custody -> report) genuinely happens on screen when you run it — not "should happen," actually happens, watched.
- **Done when:** the report is light-themed and reads like a real Section 63 certificate with a blank signature line, the job log shows every kind of job, Settings and Evidence catalog have nothing fake left in either of them, and the demo script runs start to finish without anyone intervening.

5. How this actually gets judged

A 3x official SIH evaluator's own advice, in plain terms: AI coding tools mean anyone can put together a working-looking screen in an afternoon, so just having a demo run isn't worth much anymore — evaluators weight it at roughly 20–30% of the score now, not the 60% it used to be. The rest is architecture, whether the code is actually organized sensibly, and whether the person presenting a piece of it can explain how it works, not just click through it. In practice: everyone here should be able to explain their own part from memory, not read it off a slide.

Five questions worth having a real answer ready for:

1. Is your input data real or synthetic? — Real Dahua .dav, a real Hikvision NVR export, real Digital Corpora .E01 disk images. Synthetic only where it's a filesystem-structure fixture built for testing, and that gets said plainly, not hidden.
2. What's the AI actually doing that a simpler rule-based system couldn't? — Answer per feature, not in general. Matching a person's appearance across cameras genuinely needs a learned model — there's no rule for that. Motion and scene-change detection are plain image comparison, not AI. The proximity flag is just checking whether two boxes overlap. Don't call the non-AI parts AI.
3. What happens when something's missing or fails? — It says so. A model that isn't installed skips with a clear message instead of crashing. Damaged recordings get marked partial instead of silently dropped. Confidence gets stated, not assumed.
4. Would this hold up on real production-scale data? — Honestly: it's been checked against real individual devices, not a fleet of them. Say that directly instead of overselling it.

5. How do you know the hash record itself wasn't changed after the fact? — Each custody entry's hash depends on the one before it, so changing an old entry breaks every entry that comes after it — that's what "append-only, hash-chained" means in practice. This is also a direct answer to the "Blockchain & Cybersecurity" theme this problem statement is filed under, and it should be said outright in the report and the pitch, not left implicit.

On claiming support for all 8 vendors: don't. The problem statement itself says teams realistically go deep on one or two vendors, not all eight — which is already the shape of what's built here (Dahua, Hikvision, and Honeywell parsed properly; the rest get acquisition and a generic recovery attempt, clearly labeled as such). Saying that plainly up front is a stronger position than a claim that falls apart the first time someone asks to see an eighth vendor live.

The presentation slides follow the standard SIH format: Problem Understanding, Tech Stack & Architecture, Feasibility/Risks & Mitigation, Real-World Impact, Research & References. The architecture slide needs to show real backend structure, not a screenshot of the UI. The references slide should cite the same sources already backing the Hikvision engine (a 2015 forensics conference paper, a 2023 Journal of Forensic Sciences paper, a 2025 paper in the journal Information) — they're already written down in docs/reference/hikvision_fs.md; they just need to be copied onto a slide.

One note on the AI models, in case other material on this problem statement suggests otherwise: Pramaan runs small, CPU-only models (YOLOX-nano for detection, a compact re-identification model, all through OpenCV, no PyTorch), on purpose — it's a deliberate choice to keep compute cost and install size low, because judges take that seriously. If you've seen a plan for this problem statement that uses a heavier stack (like YOLOv11 with a full tracking and face-recognition pipeline), that's a different team's approach, not this one — don't switch to it.

What other public teams have built for this exact problem statement: checked the four other public GitHub repos for SIH26150. One (MIT-licensed, called "24fps") is a real, disciplined build — genuine byte-level parsing for one vendor, real ML inference, deterministic multi-signal cross-camera correlation (which is where the verdict idea above came from), and reproducible PDF reports. Nothing was copied from it — the ideas above were rebuilt independently in Pramaan's own code and style. The other three aren't real competing attempts: one's actual code solves an unrelated hackathon problem with the title swapped, one is mostly empty folders with an admitted "no active forensic algorithms yet," and one doesn't even start up (a broken import in its only backend service). None of the four have any real DVR/NVR sample footage or disk image sitting in the repo that we could use for testing.

6. Before calling anything done

- `python -m pytest engine/tests -q` passes.
- Frontend typecheck, build, and format check all pass on whatever you changed (`npm run build`, `npx tsc`, `npm run format:check`).
- The "done when" line for your section, shown working on real evidence — not a fixture, not on the first attempt, not described as "should work."
- Nothing you touched leaves a placeholder, a fake number, or a feature that only works in one specific case behind. If it's not really done, it's not in the demo.